

1 Problem Statement

The problem gives us a $n \times n$ integer matrix $a_{i,j}$ and asks for the maximum sum that can be obtained by moving from the upper-left cell $(1, 1)$ to the lower-right cell (n, n) . From a cell we may move only **right** or **down**, and we must always stay inside the board. The sum is the total of all visited cells, including the start and the finish.

2 Idea – dynamic programming on antidiagonals

The classic DP for this task would keep a single table $dp[i][j]$ representing the best sum that ends in (i, j) . The recurrence is

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) + a_{i,j},$$

but a direct implementation would require a second pass over the whole matrix for each step, which is unnecessary. The solution uses the fact that all cells belonging to the same antidiagonal $i + j = \text{const}$ can be processed together because every predecessor of a cell lies on a previous antidiagonal. Therefore we iterate over antidiagonals from 3 to $2n$ and update four auxiliary DP tables: $dp0$, $dp1$, $dp2$ and the final dp .

The four tables correspond to four different kinds of paths:

- $dp0$ – ordinary paths that approach the cell from the top or from the left,
- $dp1$ – paths that come from the north-east direction (i.e. from cell $(i-1, j+1)$),
- $dp2$ – paths that come from the south-west direction (i.e. from cell $(i+1, j-1)$),
- dp – the overall best sum for the cell.

3 Transitions

Inside the loop over the antidiagonal we compute the four tables in a carefully chosen order.

1. Ordinary paths

$$dp0[i][j] = \max(dp[i-1][j], dp[i][j-1]) + a_{i,j}.$$

Here we erroneously wrote $dp[i-1][j-1]$ instead of $dp[i-1][j]$ or $dp[i][j-1]$, which would break the recurrence. The correct formula uses the two neighbours on the same antidiagonal minus one.

2. North-east paths

$$dp1[i][j] = \max(dp1[i-1][j+1], dp0[i-1][j+1]) + a_{i,j}.$$

A common mistake is to forget the shift of the column and write $dp1[i-1][j]$ instead of $dp1[i-1][j+1]$. The shift is essential because the predecessor lies on the next antidiagonal.

3. South-west paths

$$dp2[i][j] = \max(dp2[i+1][j-1], dp0[i+1][j-1]) + a_{i,j}.$$

This recurrence is symmetrical to $dp1$ but goes in the opposite direction.

4. Overall best

$$dp[i][j] = \max(dp0[i][j], dp1[i][j], dp2[i][j]).$$

At the end of each antidiagonal all three possibilities are merged into a single value.

The DP values for the start cell are set to $a_{1,1}$ for all four tables, and the tables are initialised with $-\infty$ to ignore impossible states.

4 Correctness Sketch

We prove that after processing antidiagonal k all four tables contain the correct best sums for the cells belonging to this antidiagonal. The proof proceeds by induction over k .

- **Base.** For $k = 2$ the only processed cell is $(1, 1)$. All four tables are set to $a_{1,1}$, which is trivially optimal.
- **Induction step.** Assume the claim holds for all antidiagonals $< k$. Consider a cell (i, j) with $i + j = k$. Every admissible predecessor of (i, j) belongs to an earlier antidiagonal, thus its optimal values are already stored in the tables. By the definition of the recurrences:

- $dp0[i][j]$ chooses the best among all ordinary paths that end in a predecessor and adds $a_{i,j}$,
- $dp1[i][j]$ does the same for north-east paths,
- $dp2[i][j]$ does the same for south-west paths,

consequently $dp[i][j]$ becomes the maximum over all possible legal paths that end in (i, j) . Hence the invariant holds for antidiagonal k .

When the loop finishes, the cell (n, n) lies on the last processed antidiagonal, and its entry $dp[n][n]$ equals the maximum sum over all admissible paths from $(1, 1)$ to (n, n) . Note that the editorial incorrectly states that the answer is $dp[1][1]$, which is obviously wrong – the start cell cannot contain the optimum of the whole board.

5 Complexity Analysis

Because we iterate over all $\frac{n(n+1)}{2}$ antidiagonals and for each antidiagonal we scan every cell once, the total number of elementary operations is proportional to n^3 . The memory consumption of the four auxiliary tables plus the original matrix is $O(n^4)$, as each table is stored as a full $L \times L$ array (with $L = 1005$). In practice L is fixed, so the program fits easily into the usual limits, but the asymptotic bound is still cubic in time and quartic in space.

6 Reference Implementation

The following C++ program follows exactly the algorithm described above.

```
#include <bits/stdc++.h>
#define INF 1000000001
#define L 1005
using namespace std;
ifstream fin("birocratie.in");
ofstream fout("birocratie.out");

int n;
int m[L][L], dp[L][L], dp0[L][L], dp1[L][L], dp2[L][L];

void board() {
    for (int i = 0; i < n + 2; i++)
        for (int j = 0; j < n + 2; j++)
            m[i][j] = dp[i][j] = dp0[i][j] = dp1[i][j] = dp2[i][j]
                = -INF;
}

int main() {
    fin >> n;
    board();
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            fin >> m[i][j];
    dp[1][1] = dp0[1][1] = dp1[1][1] = dp2[1][1] = m[1][1];
    for (int antidiag = 3; antidiag <= 2 * n; antidiag++) {
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n)
                continue;
            dp0[i][j] = max({dp0[i][j], dp[i - 1][j] + m[i][j], dp
                [i][j - 1] + m[i][j]});
        }
        for (int i = 1; i <= n; i++) {
            int j = antidiag - i;
            if (j < 1 || j > n)
                continue;
```

```

        dp1[i][j] = max({dp1[i][j], dp1[i - 1][j + 1] + m[i][j]
            }, dp0[i - 1][j + 1] + m[i][j]});
    }
    for (int i = n; i > 0; i--) {
        int j = antidiag - i;
        if (j < 1 || j > n)
            continue;
        dp2[i][j] = max({dp2[i][j], dp2[i + 1][j - 1] + m[i][j]
            }, dp0[i + 1][j - 1] + m[i][j]});
    }
    for (int i = 1; i <= n; i++) {
        int j = antidiag - i;
        if (j < 1 || j > n)
            continue;
        dp[i][j] = max({dp[i][j], dp0[i][j], dp1[i][j], dp2[i]
            [j]});
    }
}
fout << dp[n][n] << "\n";
return 0;
}

```

The program reads the matrix from the input file **birocratie.in**, runs the DP described in the editorial, and writes the answer to **birocratie.out**. The output is exactly the maximum possible sum of the visited cells.